

Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Organización de Lenguajes y Compiladores II.
Ing. Mario Tobar.



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

PIGLATIN - MANUAL TÉCNICO

Estudiante	Carné
201931811	Diego José Maldonado Monterroso

Quetzaltenango, Guatemala.

Lunes 24 de agosto de 2026.

TECNOLOGÍAS

- **Java 21.**

Se compila con el JDK 26 pero se apunta a 21, por compatibilidad con Java FX.

- **Maven 3.9.11.**

Gestión de dependencias y build.

- **ANTLR 4.13.2.**

Genera el lexer y el parser a partir de ***LatinLexer.g4*** y ***LatinParser.g4***.

Configuración:

```
<configuration>
  <sourceDirectory>${basedir}/src/main/antlr4</sourceDirectory>
  <listener>false</listener>
  <visitor>true</visitor>
</configuration>
```

Esta configuración permite construir el AST con el ***Visitor*** generado.

- **JavaFX 21.0.9.**

Interfaz gráfica.

- **RichTextFX 0.11.3.**

Editor de código con resaltado de sintaxis

PALABRAS RESERVADAS

Marcadores de Sección.

Palabra	Significado
VARIABLES>	Abre la sección de variables globales (opcional)
MUNERA>	Abre la sección de funciones (opcional)
MAIOR>	Abre la funcion principal (obligatoria)
VARIABLES[...]	Sección de variables locales, al inicio del cuerpo de una función
FINIS;	Cierra el programa (en mayúsculas)

Declaración.

Palabra	Significado
esto	Declara una variable o un atributo
series	Declara un arreglo
estructura	Define una estructura del usuario
finis	Cierra una estructura, un condicional o un ciclo (en minúsculas)

Tipos.

Palabra	Tipo	Jerarquía
textum	Cadena de texto	5
decimalis	Número con decimales	4
numerus	Número entero	3
littera	Carácter	2
bool	Booleano	1
verum	Literal verdadero (y tipo, en la forma especial)	1
falsus	Literal falso (y tipo, en la forma especial)	1

Control de flujo.

Palabra	Significado
si	Condicional
aliter	Rama alternativa (con o sin condicion)
dum	Ciclo mientras
facere ... dum	Ciclo hacer-mientras
per	Ciclo con iterador
perge	Continuar (continue)
interrumpe	Interrumpir (break)

Funciones.

Palabra	Significado
actio	Función sin retorno
ratio	Funcion con retorno
reddere	Retornar un valor

Logica.

Palabra	Significado
non	Negación lógica

En total tenemos 27 palabras reservadas.

SÍMBOLOS Y OPERADORES

Aritméticos.

Símbolo	Operación	Tipos aceptados
+	Suma / concatenación	numéricos, o cualquiera con textum
-	Resta	solo numéricos
*	Multiplicación	solo numéricos
/	División	solo numéricos

Relacionales.

Símbolo	Operación	Tipos aceptados
==	Igual	tipos iguales (textum y bool incluidos) o dos numéricos
!=	Distinto	ídem
< > <= >=	Comparación	solo numéricos

Lógicos.

Símbolo	Operación	Tipos
&&	Conjunción	ambos bool
	Disyunción	ambos bool
non	Negación	bool

Incremento y decremento.

Símbolo	Operación	Tipos
++	Suma una unidad	numerus, decimalis
--	Resta una unidad	numerus, decimalis

Entrada, salida y asignación.

Símbolo	Operación	PigLatin
>>	Imprimir	%OINK
<<	Leer	%OINK_OINK
=	Asignar	(no cambia)

Agrupación y separadores.

Símbolo	Uso
()	Agrupar expresiones, parámetros, argumentos, condiciones
{ }	Bloques, cuerpos, literales compuestos
[]	Índices, tamaños, sección VARIABLES[]
;	Fin de instrucción
:	Separa el nombre del tipo
,	Separa parámetros, argumentos y campos
.	Acceso a un atributo

Comentarios.

Forma	Sintaxis
De línea	// ...
De bloque	/* ... */

Van al canal HIDDEN: el parser los ignora, pero siguen disponibles para el coloreado.

Un comentario de bloque **sin cerrar** no es un comentario: lo reconoce la regla de error COMENTARIO_SIN_CERRAR y se reporta como error léxico.

Lo que el lenguaje NO reconoce.

Cuatro reglas cierran el lexer. Las tres primeras nombran el error clásico del literal o el comentario que nunca cierra, para darle un mensaje propio en vez de reportar solo la comilla suelta; la cuarta es el cajón de sastre final.

Regla	Qué captura	Mensaje
TEXTO_SIN_CERRAR	"sin cerrar	<i>Cadena sin cerrar: falta la comilla doble final.</i>
CHARACTER_SIN_CERRAR	'a	<i>Caracter sin cerrar: falta la comilla simple final.</i>
COMENTARIO_SIN_CERRAR	/* nunca cierra	<i>Comentario de bloque sin cerrar: falta '/*.*</i>
CHARACTER_INVALIDO	#, @, \$...	<i>Simbolo no reconocido por el lenguaje.</i>

Las cuatro se pintan de **rojo subrayado** en el editor y **cortan el pipeline**: con un error léxico no se intenta parsear.

Por qué no rompen lo bien formado. Van declaradas **después** de su versión cerrada, y ANTLR se queda con la coincidencia más larga; la versión cerrada siempre gana por al menos un carácter:

Entrada	Token que gana
"abc"	TEXTO
'a'	CARACTER
/* a */	COMENTARIO_BLOQUE
6 / 2	DIVISION — /* exige que el * sea el carácter siguiente

PRECEDENCIA DE OPERADORES

De menor a mayor:

Nivel	Operadores	Asociatividad
1		izquierda
2	&&	izquierda
3	== !=	izquierda
4	< > <= >=	izquierda
5	+ -	izquierda
6	* /	izquierda
7	non, - unario, ++/-- prefijos	derecha
8	[], ., (), ++/-- sufijos	izquierda
9	literales, identificadores, (expresión)	—

ESTRUCTURA DE UN PROGRAMA

```
VARIABLES>           // opcional: variables, arreglos y estructuras globales
...

MUNERA>              // opcional: solo definiciones de funciones
...

MAIOR>               // OBLIGATORIA: la función principal
...

FINIS;
```

Dónde se puede declarar una variable:

Lugar	¿Permitido?
VARIABLES>	Sí
VARIABLES[] de una función	Sí
Nivel superior de MAIOR>	Sí
Inicialización de un per	Sí
Dentro de un si, dum, facere o del cuerpo de un per	No

SINTAXIS POR CONSTRUCCIÓN

Variables.

```
esto mi_entero : numerus 10 ;  
esto total     : numerus fuerza * 2 ;  
esto nombre    : textum "Somos la resistencia";  
esto gravedad  : decimalis 9.81;  
esto inicial   : littera 'a';  
esto activo    : bool verum;  
esto heredado  : verum;           // forma especial: el tipo es el valor  
esto resultado : numerus = calcularPoder(10, 0.5); // el '=' es opcional
```

Arreglos.

```
series mis_enteros[2] : numerus {1, 1}; // con valores iniciales  
series mis_enteros_[2] : numerus;       // sin inicializar  
series nombres[2]     : textum {"Hola", "Adios"};  
series valores[2]     : {verum, falsus}; // sin tipo: se deduce del primer valor
```

Las posiciones **empiezan en 0**. El índice puede ser una expresión; si el compilador puede evaluarla, valida el rango.

Estructuras.

```
structura Persona {  
  esto nombre : textum;  
  esto edad  : numerus;  
} finis;
```

Instanciación — **el orden de los atributos no importa**, pero **todos** deben tener valor:

```
esto mi_personaje : Persona {  
  nombre: "Yennifer",  
  edad: 999  
}
```

Esa declaración **no lleva ;**; termina con el }.

Anidadas, con un campo arreglo:

```
structura Animal {
    esto nombre: textum,
    esto apodo: textum
} finis;

structura Selva {
    esto valido : verum,
    series animales : Animal    // sin tamaño: la dimensión llega al instanciar
} finis;

esto mi_selva: Selva {
    valido: verum,
    animales: Animal[7]        // aquí sí, con tamaño
}

mi_selva.animales[1] = {
    nombre: "Perro",
    apodo: "Canis"
}
```

Condicionales.

```
si (x > 10 && y < 5) { ... } finis;
```

```
si (x > 10) { ... } aliter { ... } finis;
```

```
si (x > 10) { ... } aliter (x > 5) { ... } aliter { ... } finis;
```

La condición debe ser **estrictamente booleana**.

Ciclos.

```
dum (x < 100) { x = x + 1; } finis;
```

```
facere { ... } dum (x < 10);
```

```
per (esto i : numerus 0; i < 10; i++) { ... } finis;
```

perge e interrompe **solo** dentro de un ciclo.

Funciones.

```
actio atacarCerdos(esto fuerza : numerus, esto precision : decimalis)
{
  VARIABLES[
    esto intentos : numerus 0;
  ]
  ...
} finis;

ratio numerus calcularPoder(esto fuerza : numerus)
{
  VARIABLES[
    esto total : numerus fuerza * 2;
  ]
  reddere total;
} finis;
```

Reglas: el retorno debe ser del tipo declarado, **todos los caminos** deben retornar, y no puede quedar código inalcanzable.

Entrada y salida.

```
>> "Holaaaaa";
>> mi_string;
>> mi_string >> mi_textum; // varias en una instrucción

<< // lee y descarta
mi_textum << // lee y guarda
```

La lectura es la única instrucción que no termina en ;.

LEXER Y PARSER

[Lexer.g4](#)

[Parser.g4](#)

TRADUCCIÓN A PIGLATIN

Elemento	¿Se traduce?	Ejemplo
Identificadores	Siempre	fuerza → uerzafay
Palabras reservadas	Siempre	actio → actioway, dum → umday
Marcadores de sección	Siempre	VARIABLES> → ARIABILESVAY>, FINIS; → INISFAY;
Nombres de estructura	Siempre	Persona → Ersonapay
Símbolos ({ ; (: ...)	Nunca	<i>"Los símbolos usados en el idioma no cambian"</i>
Interior de un textum	Opcional	Casilla en la pestaña PigLatin; el enunciado no obliga

Ley de consonantes.

Si el identificador empieza con una o más consonantes, se mueven al final y se añade **ay**.

Entrada	Salida
fuerza	uerzafay
tabla	ablatay

Ley de vocales.

Si empieza con vocal, solo se añade **way**.

Entrada	Salida
inicio	inicioway
archivo	archivoway

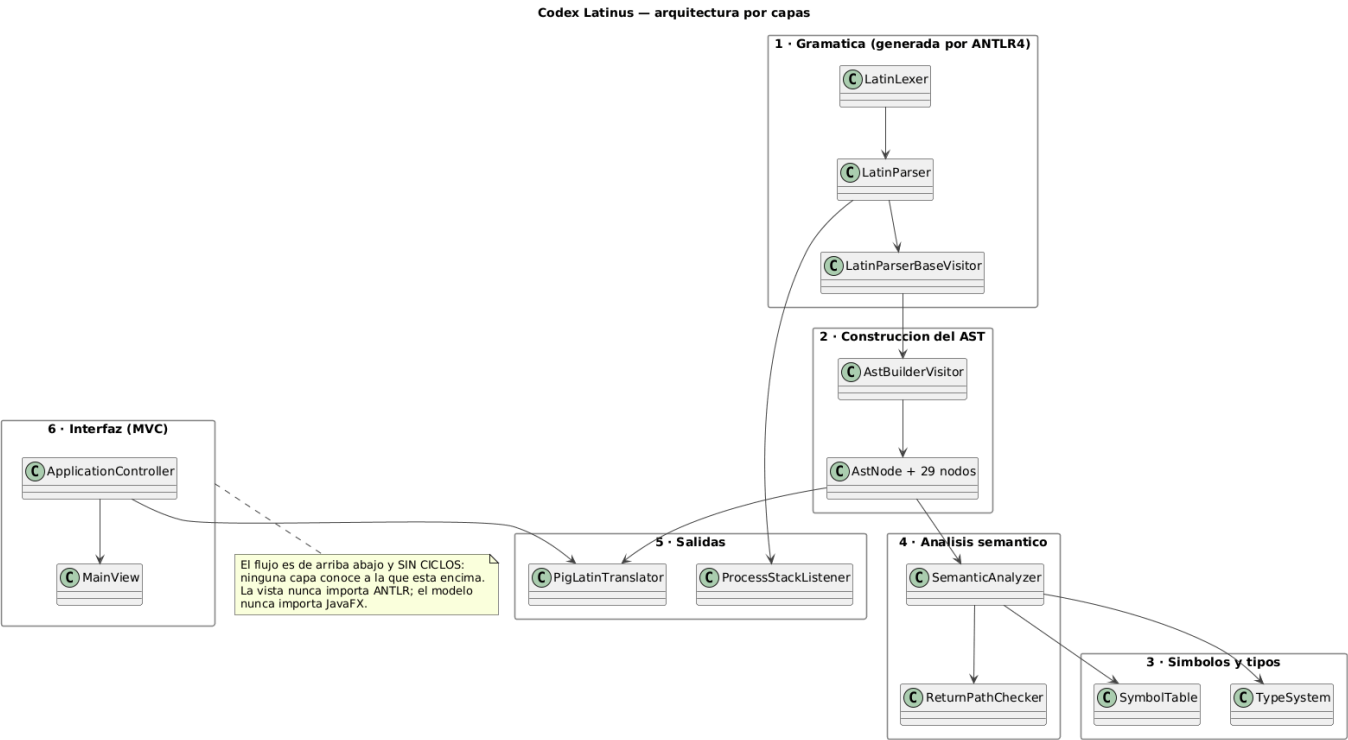
Ley porcina.

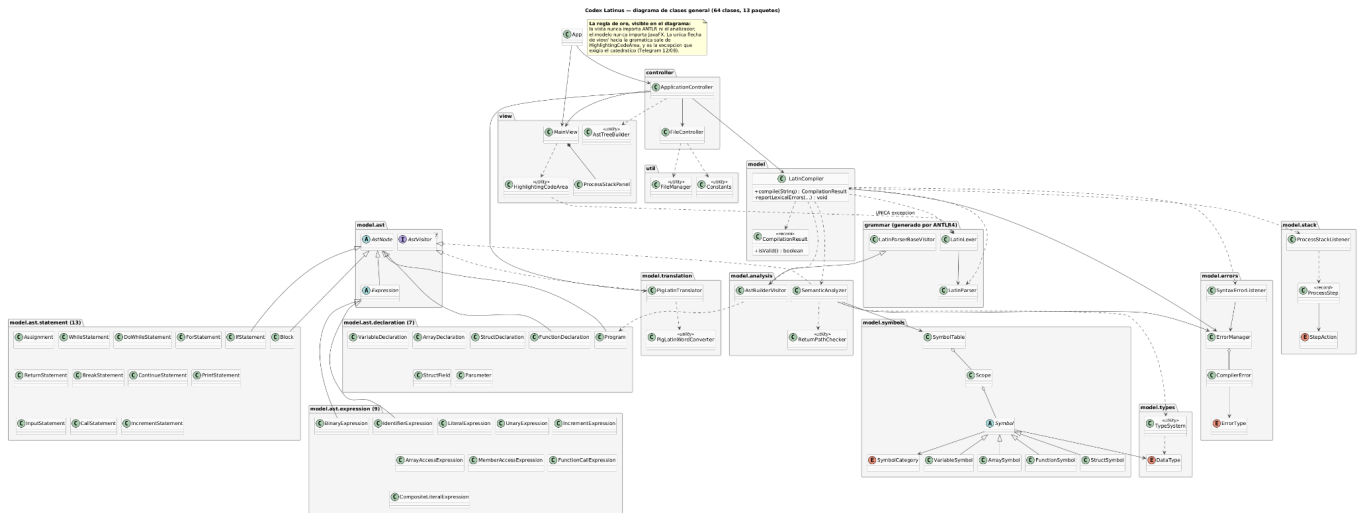
Función	PigLatin
Leer (<<)	%OINK_OINK
Imprimir (>>)	%OINK

Reglas complementarias.

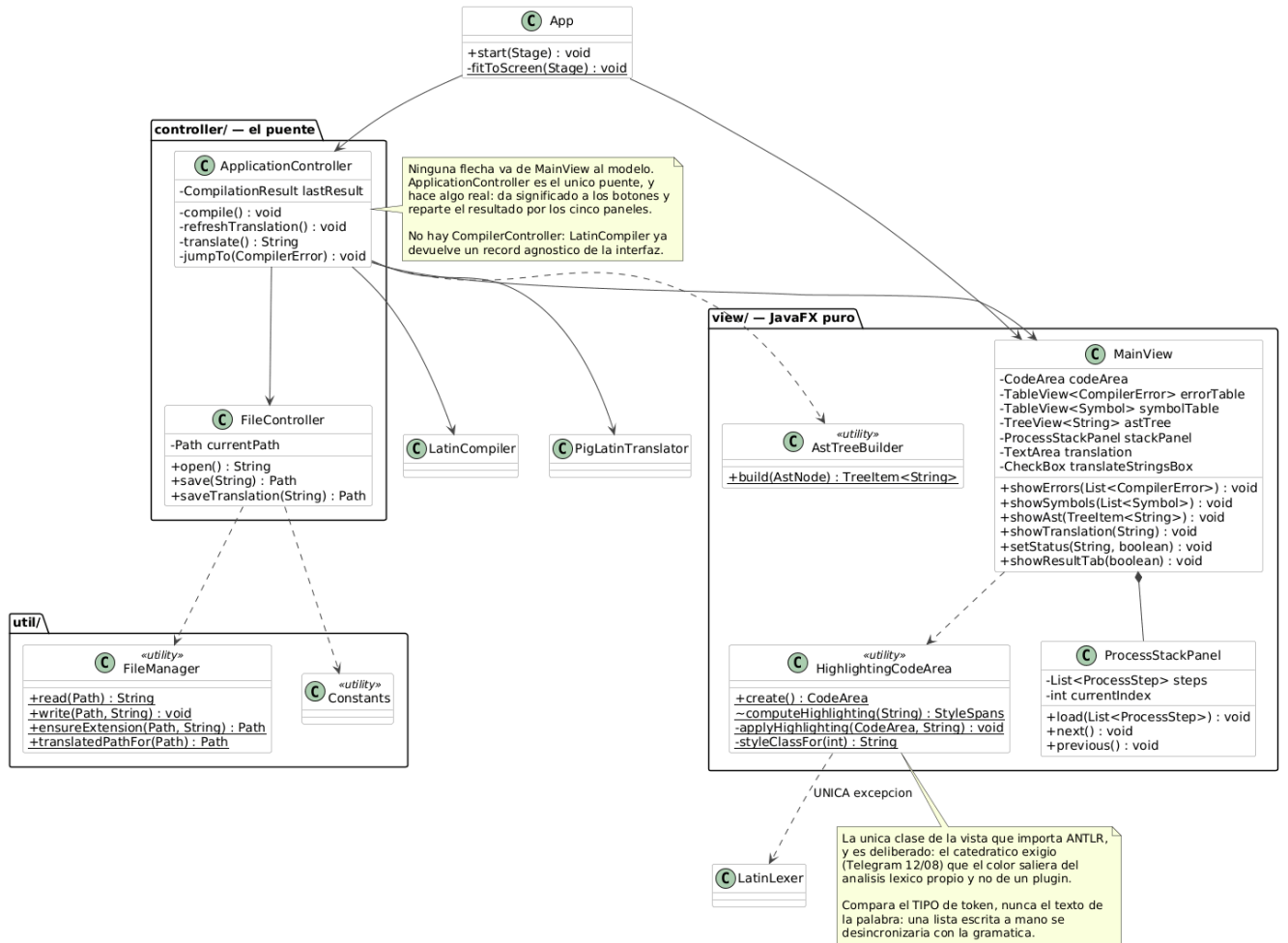
Caso	Resultado	Razón
Sin vocales (xyz)	xyzay	Ninguna ley aplica; no se lanza excepción
_ y dígitos (mi_variable2)	imay_ariablevay2	<i>"Los símbolos usados en el idioma no cambian"</i>
PascalCase (Persona)	Ersonapay	Se conserva el estilo de capitalización
camelCase (calcularPoder)	alcularPodercay	ídem
Mayúsculas (VARIABLES)	ARIABLESVAY	ídem
Símbolos ({ ; ;)	sin cambio	Explícito en el enunciado

DIAGRAMAS

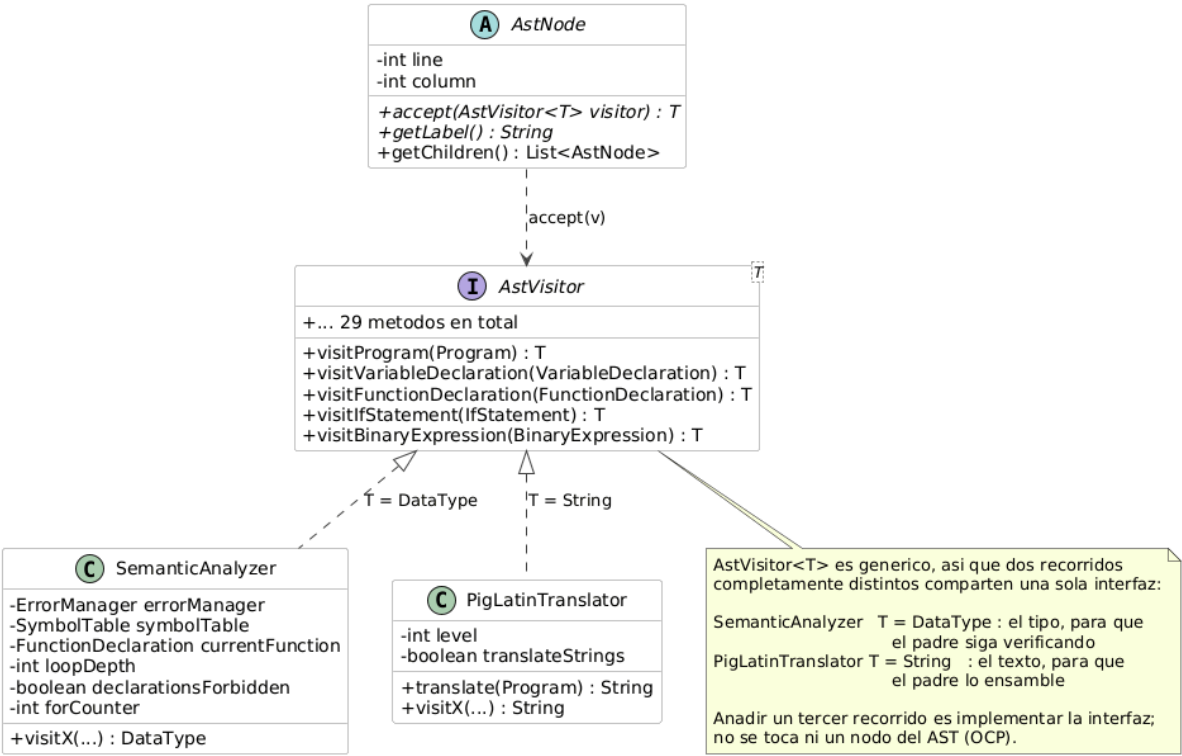




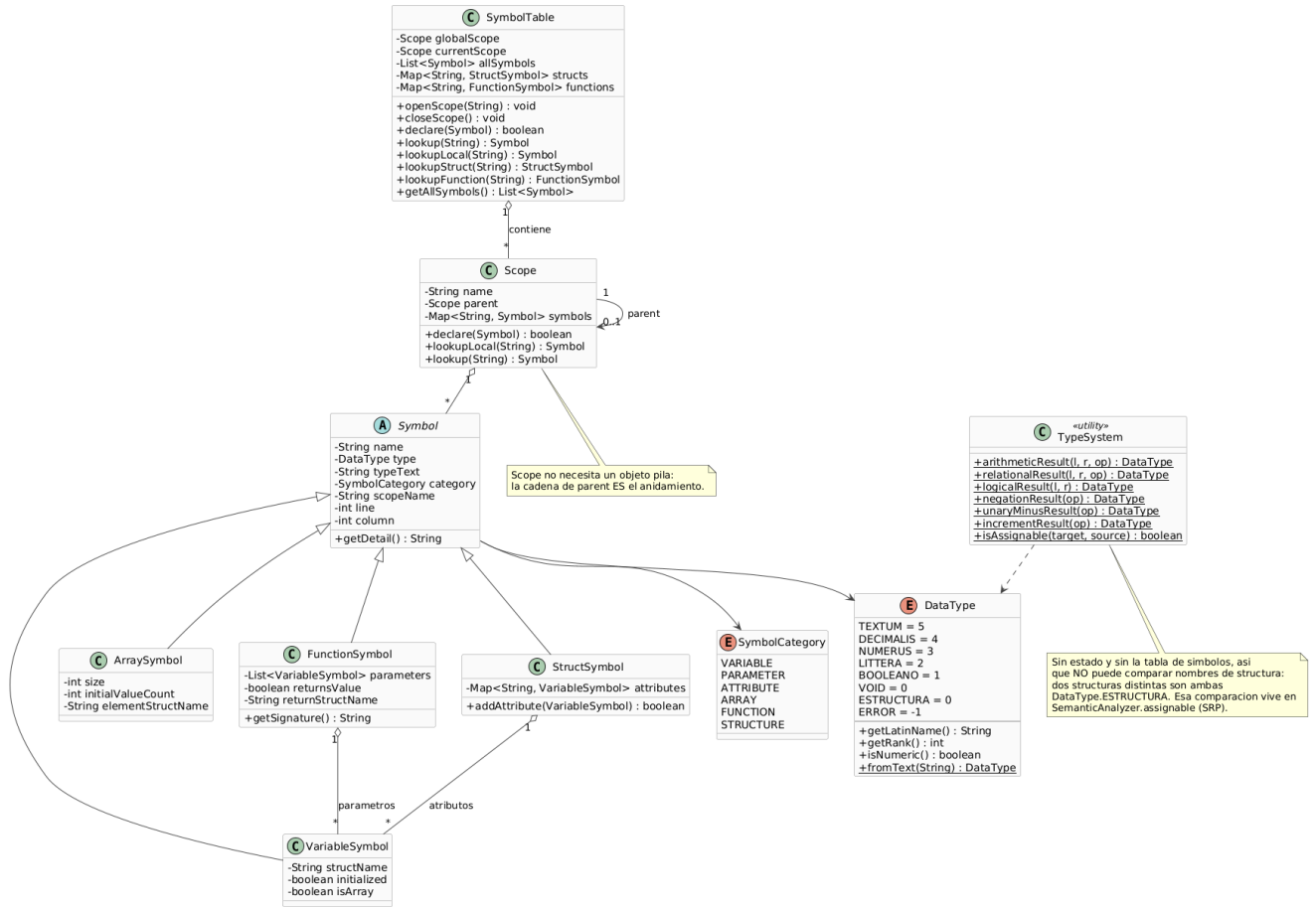
Interfaz (MVC) — la vista nunca toca el modelo



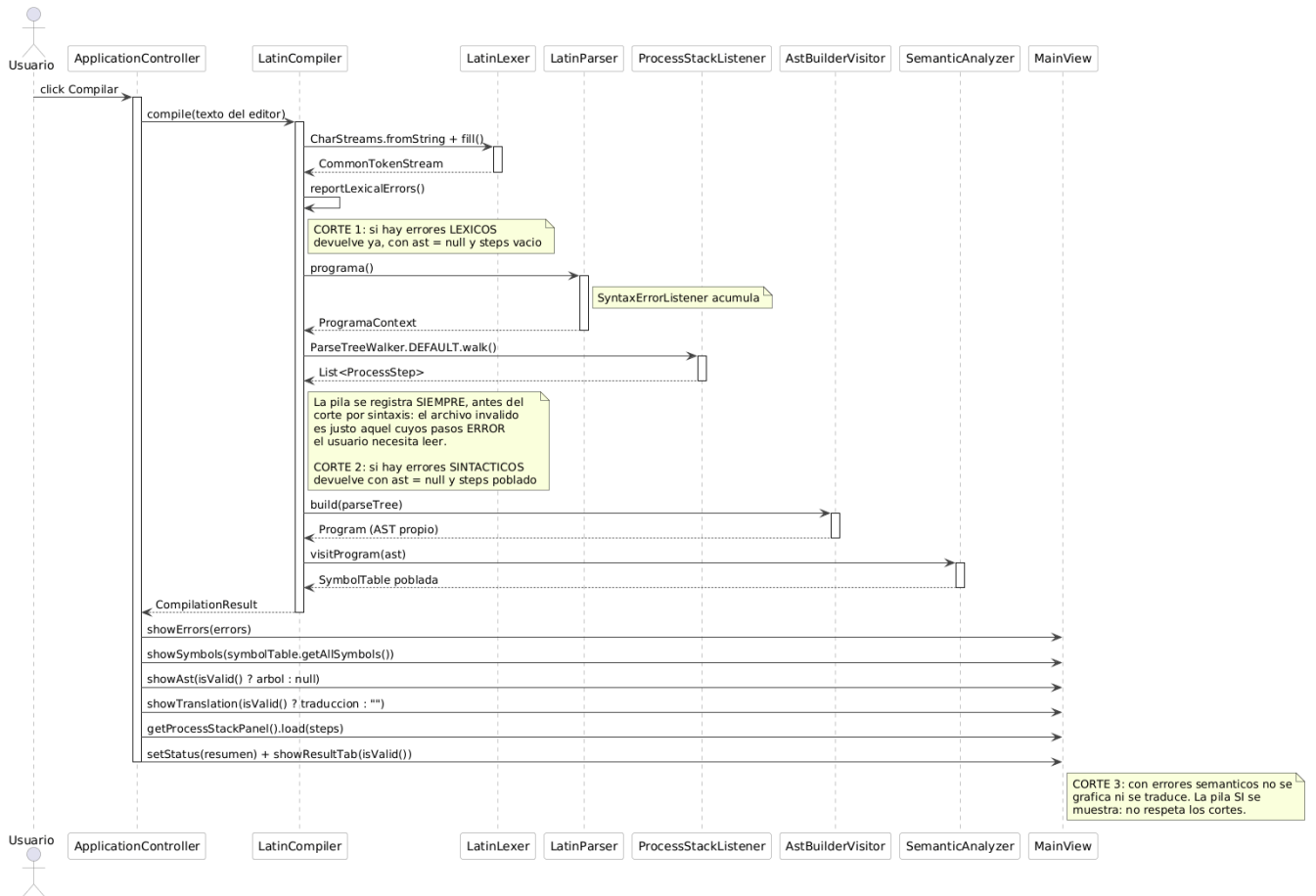
Patron Visitor — dos recorridos sobre la misma interfaz



Simbolos, ambitos y sistema de tipos



Flujo Compilar — del clic a los cinco paneles poblados



Pipeline de compilacion — texto a PigLatin, con los 3 cortes

This syntax is deprecated, you must add <<#FF0000>> at the end of the line, after the ';'.

